NETMIND

Project Technical Study and Team Guide

Detailed explanation of the simulator, algorithms, data structures, OOP design, Python analysis, API, frontend, and GraphRAG.

Purpose. Teammate handoff and oral-presentation guide explaining how components fit together, what each owns, and why the selected data structures and design patterns make sense.

Scope: Describes source currently present in the workspace, with the implementation guide and responsibility document as context. No fresh build or test run was performed while preparing this PDF. Where older docs differ, current source is described and differences are called out.

Reviewed: 22 October 2026
Project: NetMind - C++ network simulation and analysis workbench
Stack: C++17, CMake, Catch2, nlohmann JSON, Python, FastAPI, React/Vite, NetworkX, LangGraph, Groq

Read Sections 1-3 for the overview; Sections 4-16 for technical detail; finish with status and viva questions.

1. What the project does

NetMind is a discrete-event network simulator and analysis workbench. A user defines a topology and scenario, selects routing behavior, adds traffic and time-based network events, runs the simulation, and inspects outputs. The simulator models packet forwarding, transmission time, propagation delay, queues, loss, failures, restoration, temporary congestion, and bursts. It turns events into traces and metrics, then provides browser review, Python analysis, and graph-grounded questions.

Topology versus scenario

A **topology** says what the network is: devices, addresses, and links. A **scenario** says what happens in one experiment: topology, algorithm/weights, traffic, failures or congestion, random seed, and duration. The engine advances simulated time by processing scheduled events instead of waiting in real time.

What users can learn

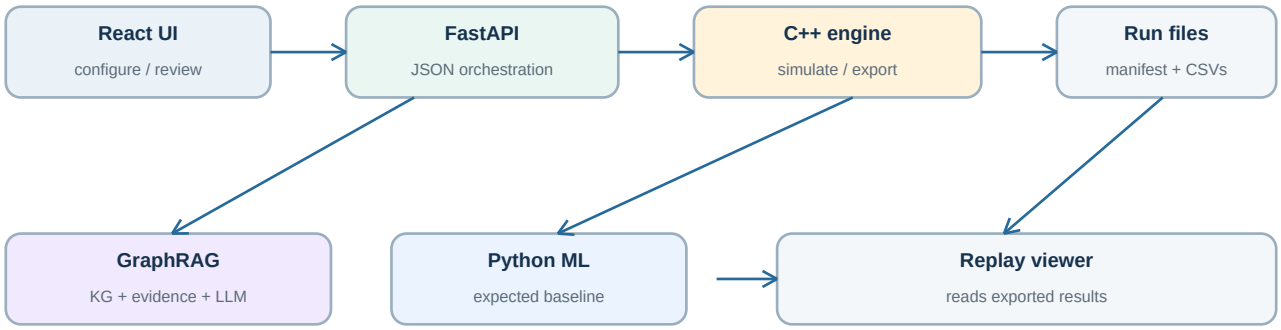
- How graph structure and weights affect routes, and why BFS, DFS, Dijkstra, and Bellman-Ford differ.
- How packets experience serialization delay, propagation delay, queueing, loss, TTL expiry, failure, and delivery.
- How to compare latency, throughput, goodput, reliability, drops, queue length, utilization, and recovery time.
- How evidence connects: event trace, aggregate reports, replay, expected metrics, and graph-linked explanations.

Baseline terminology: ML predicts expected metrics from similar historical simulation runs. It does not calculate an optimal route. A deviation flag means "different from the fitted reference," not "proven anomaly."

Output	Meaning
Topology JSON	Static nodes, links, and link parameters.
Scenario JSON	Topology plus routing, traffic, events, duration, and seed.
Run manifest	Run identity and metadata tying settings to results.
Trace CSV	Ordered packet and network events.
Flow/link/window/network CSVs	Aggregated measurements at multiple scopes.

2. Architecture and data flow

The system combines a C++ simulation core, Python analysis/API, and React browser interface. FastAPI orchestrates the CLI and files. The replay viewer consumes run data. ML and GraphRAG analyze exported results. This separation keeps simulation mechanics independent from rendering and LLM prompts.



The viewer reads run files; it is not driven by formatted CLI text.

One run, end to end

- User loads a topology/scenario and configures the experiment.
- React sends structured JSON to FastAPI; it does not scrape terminal output.
- API materializes/invokes the scenario and calls the C++ CLI.
- SimulationEngine processes events and notifies observers; trace and metrics create files.
- API reads those outputs and returns run IDs, metrics, and file references.
- Baseline compares expected/measured metrics; replay visualizes events; GraphRAG retrieves evidence and can produce a cited answer.

The CLI remains useful independently for scripts and tests. Python can inspect outputs without linking into C++. GraphRAG explains recorded evidence without changing the simulation.

3. Build system and common foundations

Build and test setup

C++ uses CMake/C++17 and Ninja presets (debug, release, sanitizer). The core library is separate from the optional CLI. Catch2 and CTest run C++ tests; nlohmann JSON reads config. Python uses pytest; frontend has npm scripts.

Example checks (guidance only; not run while preparing this guide):
cmake --preset debug
cmake --build --preset debug
ctest --test-dir build/debug --output-on-failure
python -m pytest -q
cd frontend_v2 && npm run lint && npm run build

Shared types, units, errors, IPs

Types.h centralizes ID aliases, simulation time, enums, direction, channel identity, and string conversion. Named aliases make intent clear even when underlying values are integers. **Units.h** centralizes validated conversions for milliseconds, transmission time, and Mbps. **Errors.h** distinguishes configuration from topology errors. **Ip.h** validates, parses, formats, and compares IPv4 addresses/subnets.

Randomness

Rng wraps a seeded Mersenne Twister. A fixed seed repeats stochastic behavior such as loss and traffic; use the same scenario and seed while varying algorithm for a controlled comparison.

Area	Purpose
include/netmind + src	C++ model, graph, routing, simulation, metrics, CLI.
apps	Executable entry points.
tests	C++ tests, fixtures, golden data, benchmark source.
python	IO, dataset generation, baseline, GraphRAG, API.
frontend_v2	React/Vite and replay viewer.
config/docs/contracts	Examples and shared schema agreements.
data	Generated run outputs and datasets.

4. OOP design and module contracts

OOP gives each component a responsibility and makes alternate behaviors replaceable. Interfaces are used where several implementations share an operation; composition assembles runtime collaborators.

Abstraction	OOP idea	Purpose
NetworkEntity	Abstract base + virtual destructor	Common identity and safe polymorphic cleanup.
Node	Abstract base	Shared state; Host/Router/Switch implement receivePacket.
RoutingAlgorithm	Strategy interface	Four algorithms share route/table calls.
WeightPolicy	Strategy interface	Change cost rules without rewriting algorithms.
TrafficPattern	Strategy interface	Constant, bursty, Poisson behavior plugs into Flow.
SimObserver	Observer	Trace and metrics listen without engine knowing internals.
NetworkEvent	Polymorphic scheduled command	Failures, restoration, congestion, bursts apply uniformly.
SimContext	Facade	Nodes request delivery/transmission without engine dependency.

Ownership and composition

Graph uniquely owns nodes/links with unique_ptr: exclusive ownership, clear lifetime, stable heap addresses. Flow owns its TrafficPattern. Session owns engine and observers; SimulationEngine observes them non-owningly, so Session must keep them alive. Router has a table; Flow has a pattern; engine has graph, event queue, channels, routing manager, and observers.

Polymorphism

The engine calls receivePacket through a Node contract, dispatching to Host or Router. RoutingManager calls computeTable through a RoutingAlgorithm contract. One caller can use different implementations without type-specific orchestration. In simple terms: inheritance is "is a kind of"; composition is "has/uses"; an interface is "supports this behavior."

5. Network model

Packet and Node

Packet carries IDs, endpoints, size, type, TTL, timestamps, status, drop reason, and pathTaken. It is the self-describing work item moving through the graph. Node stores identity, name, IP, type, up/down state, and receivePacket contract. Nodes are non-copyable to avoid duplicate identities.

Host, Router, Switch

Host originates packets and consumes packets addressed to it; it can answer echo requests with reversed endpoints. **Router** checks/decrements TTL, consumes packets addressed to itself, or looks up destination and transmits/drops. **Switch** is the optional Tier 3 node subtype with switching behavior.

ForwardingTable

A hash map maps destination ID to next hop, cost, and source (dynamic/static). Lookup averages O(1). Static routes have priority and survive dynamic recomputation. A sorted-vector view gives deterministic display order.

Link

Link stores endpoints, bandwidth, latency, loss, queue capacity, state, epoch, degradation, and extra loss. It validates positive bandwidth, non-negative latency, and loss in [0,1]. Methods provide endpoint queries, effective bandwidth/loss, usability, and policy weight. Epoch changes invalidate stale callbacks.

Flow and patterns

Flow binds endpoints, rate, packet size, start/stop, and a TrafficPattern. ConstantRate derives interval from rate and size. BurstyPattern alternates active/quiet intervals. PoissonPattern samples exponential intervals. Seeded RNG injection keeps stochastic patterns reproducible.

Validation

- Unique names and valid IPv4.
- Existing link endpoints and valid link parameters.
- Hosts follow the loader edge-role contract.
- Disconnected graphs are valid, not malformed.
- TTL, routes, node/link state, and queue capacity are checked at use.

6. Graph, data structures, topology loader

Graph representation

Graph owns nodes/links in ID maps, has a name-to-ID index, and adjacency lists for neighbor lookup. Hash maps support average O(1) ID lookup; adjacency lets traversal inspect only local edges.

Operation	Structure	Typical complexity	Use
Neighbors	Adjacency list	O(degree)	Find directly connected edges.
BFS	FIFO queue + visited	O(V+E)	Reachability/min-hop traversal.
DFS	Stack/recursion + visited	O(V+E)	Deep exploration/tree routes.
Components	Union-Find	Near O((V+E) alpha(V))	Group connected nodes.
Cycles	Traversal state or DSU	O(V+E) / near-linear	Find redundant connectivity.
Dijkstra	Min-heap	O((V+E) log V)	Take smallest tentative cost.

Union-Find

Each node begins in a set. Each link unites endpoint sets. If endpoints already share a representative, that link closes a cycle. Path compression and union-by-rank keep searches shallow, making connectivity scans nearly constant-time per operation in practice.

TopologyLoader and examples

TopologyLoader parses JSON, constructs typed objects, then validates schema, duplicate IDs/names, IPs, endpoint references, and link values. Checking at the boundary lets algorithms assume graph invariants. Contracts keep C++, Python, and viewer independently compatible. Samples small4, medium10, ring_backup, dumbbell, and diamond_weighted exercise basic, larger, alternate-path, bottleneck, and weighted-path cases.

7. Routing algorithms

Routing answers which next hop a router should use. An algorithm explores the graph; a weight policy defines edge cost.

Algorithm	Mechanism	Guarantee	Typical complexity
BFS	FIFO by edge distance.	Minimum number of hops.	O(V+E)
DFS	Depth-first visited/parent state.	Valid traversal-tree route, not shortest.	O(V+E)
Dijkstra	Settle lowest tentative cost via heap.	Minimum path with non-negative weights.	O((V+E) log V)
Bellman-Ford	Relax edges repeatedly; early stop.	Weighted shortest path; detects reachable negative cycles.	O(VE)

On undirected links, a negative edge creates a negative cycle by traversing it both ways; ordinary simulation policies should stay non-negative.

Policy	Cost	Interpretation
HopCount	1 per link	Minimize hop count.
Latency	Configured link latency	Prefer lower propagation delay.
LatencyCongestion	Latency + alpha * queue delay	Penalize busy links.

RoutingManager builds tables for routers, ignores down/unusable network elements, preserves static routes when rebuilding dynamic routes, and can recompute immediately or after convergence delay. Stable tie-breaking supports repeatability. DFS and Dijkstra may agree on a given graph; demonstrate their guarantees with a topology where DFS explores a worse route first.

8. Discrete-event simulation engine

Discrete-event model

Instead of updating every packet in tiny time slices, SimulationEngine stores future events in a priority queue, jumps to the earliest event, applies it, and schedules consequences. It avoids work during intervals where nothing changes.

EventQueue and packet journey

EventQueue orders first by time and then by an increasing sequence number, resolving same-time events deterministically. It validates finite non-negative times and non-null events. A packet journey is: source transmit request; node/link checks; directional queue; serialization; loss sampling; propagation; epoch check; destination receive/deliver or another router forwarding/drop.

Channels, delays, failures

Each physical Link has independent A-to-B and B-to-A Channels. Each Channel has FIFO queue, busy state, generation, and capacity, so both directions may transmit concurrently. Serialization delay is packet bits / effective bandwidth; propagation is configured latency; queueing is time spent waiting. Loss uses the seeded RNG. Link/node epochs invalidate events scheduled before a failure. Queue flushes drop packets waiting during a failure.

Run controls

runUntil advances to a target; step processes one or several events; run processes to its stop condition. The engine exposes current time/pending events. Scenario duration validation prevents advancing beyond the configured boundary.

9. Events and scenarios

NetworkEvent packages an action behind apply(engine) and a description. This scheduled-command pattern makes failures, recovery, congestion, and bursts uniformly queueable and loggable.

Event	Effect
Link failure	Mark unusable, advance epoch, flush directional queues, request route recomputation.
Link restore	Return to service and recompute.
Router failure	Mark down, advance node epoch, invalidate affected transmissions, recompute.
Router restore	Enable future use and route computation.
Congestion	Temporarily reduce bandwidth and/or add loss, then restore later.
Traffic burst	Create temporary flow and stop generation at its end.

ScenarioManager and seed

ScenarioManager loads topology/scenario JSON, creates graph and settings, instantiates flows, resolves endpoints, schedules events, and validates times/references. One topology can be reused in many experiments. A seed controls pseudo-random sequences; keep topology, flow, events, and seed fixed while changing algorithm for controlled comparisons. A GUI request may run four algorithms, producing four run rows.

Why can a backup route still see drops? Packets already queued/in-flight can be lost, and recomputation timing matters. A backup only helps if it is connected, up, and selected when forwarding occurs.

10. Trace, metrics, reports, exports

Observer pattern

SimulationEngine notifies SimObserver listeners about sends, queueing, transmission, arrival, forwarding, delivery/drop, and network changes. TraceLogger and MetricsCollector react independently; engine logic does not depend on their internals.

TraceLogger and MetricsCollector

TraceLogger writes event-level CSV with time, type, packet/flow, location, detail. MetricsCollector accumulates per-flow, per-channel, per-window, and network state and finalizes aggregates. Time-weighted average queue is the integral of queue size over time divided by duration. Utilization compares transmitted bits to capacity-time.

Metric	Meaning
Latency	Origin-to-delivery time, summarized average/min/max/p95.
Throughput	Delivered bits per second, in Mbps.
Goodput	Useful successfully delivered payload rate.
Loss	Dropped / sent * 100.
Reliability	Delivered / sent * 100 under denominator rules.
Jitter	Variation among successive delivery latencies.
Average queue	Queue-size time integral / duration.
Utilization	Fraction of channel capacity used.
Recovery time	Time from impairment to defined recovered state.

Report selects useful summaries such as highly utilized links and flow performance. Exporters write manifest, trace, flows, links, windows, and network files. CSV names are cross-language contracts: a field mismatch can break readers or produce wrong analysis.

11. CLI

The CLI independently configures, executes, inspects, and exports simulations. It supports interactive use and scripts. The GUI uses FastAPI and run files, not formatted terminal tables.

Family	Examples	Purpose
Load/inspect	load, show topology/route/queue	Load network and inspect it.
Routing	set algo/weight, route add	Select strategy or static route.
Traffic/events	add flow, burst, fail/restore, congest	Configure workload and state changes.
Probe	ping, traceroute	Diagnostic simulated traffic/path.
Execution	run, step, reset	Advance/rebuild simulation.
Analysis	show metrics, compare, export	Inspect and save results.
Script	help, exit, --script	Discover and batch commands.

Commands implement a common interface in a registry. Longest multiword match resolves commands such as "show route"; tokenization supports quoted names and paths. Session owns shared scenario/engine/observer state without globals. Reset rebuilds runtime state; export writes run files.

Example only:
load config/topologies/ring_backup.json
set seed 42
set algo dijkstra
set weight latency
add flow PC1 PC2 5 1000 0 10

fail link R1 R2 4
restore link R1 R2 7
run 10
show metrics
export

12. Python IO and dataset generation

python/common/io.py validates expected schemas and reads manifests, traces, flows, links, windows, and network reports into typed run data. Pandas handles tabular operations. Strict checks detect producer/consumer mismatch early.

generate_dataset.py enumerates topologies, four algorithms, offered loads, event kinds, and seeds; deterministically orders/caps tasks; creates temporary scenarios; invokes the C++ CLI in isolated work directories; validates output; copies individual run files; and combines window/run information into runs_all.csv. Single-worker mode is easier to debug and uses fewer resources. A run cap does not itself guarantee every combination is covered.

runs_all.csv is generated model/evaluation rows. **windows_all.csv**, when generated, combines time windows. Per-run manifest/CSV files remain the detailed record. Recreate combined datasets if removed.

13. Expected-metric ML baseline

Features and model

The baseline estimates latency, throughput, and packet loss from topology size/connectivity, shortest-path hops and latency, bottleneck bandwidth, algorithm/event indicators, offered rate, and rate relative to capacity. RandomForestRegressor models capture nonlinear interactions.

Validation and training cadence

GroupKFold groups by topology so windows from one topology do not leak across training and validation. Out-of-fold values estimate expected metrics for unseen topologies; residual spread gives the deviation threshold. Final models are fit on the complete dataset and serialized. FastAPI loads a compatible saved model or trains from runs_all.csv. New runs get a prediction first, then are recorded. When 10 new run rows accumulate, the model is refit. If GUI runs four algorithms per request, the counter is rows/runs, not clicks.

> **Interpretation:** flag means the result differs from expected under the fitted residual rule. It does not prove fault or cause. Inspect trace and scenario.

Confidence is lower for topologies/events outside training coverage. A large but imbalanced dataset is not necessarily useful. The model learns simulator outputs and dataset bias, not real-world truth. Missing runs_all.csv must be fixed by generating training data.

14. GraphRAG and LLM

Knowledge graph and facts

build_kg.py reads manifest, topology, trace, flow, link, and window files into a NetworkX MultiDiGraph. Typed nodes include Run, Device, Link, Flow, Event, Window, sometimes Anomaly. Edges represent connectivity, run membership, forwarding, event occurrence, and flags. MultiDiGraph permits parallel/repeated time-stamped relationships. facts.py derives evidence about failures, drops, route changes, congestion, latency, and anomaly context. query.py has deterministic templates for route, algorithm comparison, congestion, event-time metrics, and fallback search.

Groq via LangGraph

The LLM path uses retrieval, generation, validation. Retrieval selects relevant graph nodes/facts; generation calls Groq; validation checks citations against retrieved evidence. `.env` supplies GROQ_API_KEY and optionally GROQ_MODEL; never commit or print the key.

Known request-size issue

A route question returned Groq HTTP 413 (request too large), likely because per-packet ROUTED_VIA edges and core evidence are not bounded by the matched-evidence cap. A proposed correction caused NameError `_num`; the inspected source did not yet contain the corrected helper. Treat compaction as unresolved: aggregate duplicate routes, rank evidence, bound all evidence, then test route questions.

Answer discipline: retrieve first, cite run evidence, distinguish observation from interpretation, abstain when evidence is insufficient.

15. FastAPI and React frontend

python/netmind_api.py exposes JSON routes for health, run discovery, scenarios, run files, GraphRAG, baseline, and GUI simulation. It validates input, invokes the simulation pipeline, reads outputs, enriches manifests, and returns structured data. It translates between browser requests and the simulator without duplicating C++ behavior.

frontend_v2 is React/Vite. The app loads topology, configures run settings, adds traffic/events, saves scenarios, triggers runs, and shows baseline, replay, algorithm comparison, and GraphRAG. Vite proxies `/api` to FastAPI. The legacy viewer reads exported run files, not CLI text.

Current-source caveat: inspected App.jsx is one scrollable page, not the requested tabs, and has no dedicated progress bar. Earlier requests do not prove changes exist in the restored source. This document reports what is present.

- Keep GROQ_API_KEY in ignored project-root .env.
- Run API and frontend separately; Vite owns the API proxy.
- Public deployment needs authentication, request limits, suitable CORS, and managed secrets.

16. Tests and contracts

C++ tests cover utilities, model validation, graph/topology, routing, event order, queues/transmission, failures/recovery/congestion, scenarios, observers, metrics, exports, and CLI. Python tests cover IO, generation, baseline, graph, facts, query, and API smoke. Frontend lint/build checks browser code. Golden fixtures stabilize examples.

Test style	Question
Unit	Does Link reject loss outside [0,1]?
Algorithm	Does Dijkstra choose lower cost on weighted diamond?
Determinism	Does same seed/scenario repeat random results?
Boundary	Does loader reject missing endpoints/invalid IP?
Integration	Does Host -> Router -> Host reach trace and metrics?
Schema/golden	Are CSV names and trace events stable?
API/UI	Does request return JSON; does frontend lint/build?

Topology, scenario, trace and CSV schemas are agreements. Update producers and consumers together. "Feature implemented," "test exists," and "test passed on this revision" are different claims; rerun from the presented commit.

17. Phase map and deferred work

Phase labels follow the internal guide; they do not imply professor approval or prove all optional items are finished.

Area	Source represented	Caveat
0 foundations	CMake, common types/utilities, contracts, test scaffold.	Rerun current build/tests.
1 model	Nodes, packet, link, tables, flows/patterns, IPv4.	Core model.
2 graph	Graph, traversals/connectivity, loader, examples.	Core graph.
3 routing	BFS, DFS, Dijkstra, Bellman-Ford, policies/manager.	Four strategies present.
4 simulation	Event engine, channels, delays/loss, failures, congestion, bursts, scenarios.	Verify integrated revision.
5 metrics/CLI	Observers, traces, reports/export, commands/scripts.	Core paths represented.
6 Python/ML	IO, generator, expected baseline and persistence.	Do not assume anomaly/forecast modules are active.

Area	Source represented	Caveat
7 GraphRAG	Graph, facts, deterministic query, Groq/LangGraph.	413 compaction unresolved in inspected source.
8 GUI	React, baseline, replay, GraphRAG.	Single page in inspected source; no tabs.
Later evaluation	Benchmark source and final integration.	Generated benchmark artifact/full status uncertain.

Deferred/needs follow-up

- Phase 6.7 plots/visual analysis.
- Phase 7.4 natural-language scenario/event creation; current GraphRAG answers questions, not arbitrary actions.
- Phase 8.3 optional enhancements, separate from core panels.
- data/bench.csv was previously deleted; do not assume it exists.
- Bound GraphRAG evidence after 413; add corrected helper and verify.
- Reconcile stale usage documentation and refresh end-to-end evidence.

18. Team responsibilities

Member	Main area	Integration role
A	Model/graph, CLI, user-facing inspection.	Network structure and entry points.
B	Routing, simulation, scenario/event execution.	Packet behavior from shared model contracts.
C	Metrics, trace, Python/ML, GraphRAG, exports/contracts.	Analysis-ready run evidence.
Shared	Integration/tests/demo.	Agree on IDs, schemas, event meaning, verification.

The Phase 0 file split was described as a suggested mapping, not a historical record. Use Git history/PRs to attribute authorship, not folder location.

19. Presentation walkthrough

- Show topology: Hosts, Routers, bandwidth, latency, backup path.
- Explain scenario algorithm, weight, seed, flow, event timing.
- Run within configured duration; explain event-driven clock.
- Compare four algorithms with other settings fixed.
- Show metrics then replay a packet and inspect its trace.
- Compare expected versus measured ML and clarify expected is not optimal.
- Ask GraphRAG an evidence-supported question and inspect citations.

Likely demo questions

- **Why drop?** Connect trace reason to TTL, route, queue, loss, or state change.
- **Why no backup?** Recompute timing, packet already in flight, backup state, and event order matter.
- **Why algorithms agree?** The graph may favor a shared route; DFS may discover it first.
- **Why exports?** Inspectable records are stable interfaces for Python, API, and viewer.

20. Professor-style viva questions

Why discrete-event?

Answer: Jump between meaningful events; avoid empty time steps.

Serialization vs propagation?

Answer: Bits/rate versus configured travel latency; queueing is separate.

Why Link and Channel?

Answer: Physical properties versus directional runtime queue/state.

Why unique_ptr?

Answer: Exclusive graph ownership and stable object addresses.

Virtual dispatch?

Answer: Node reference calls concrete Host/Router behavior.

BFS vs Dijkstra?

Answer: Fewest edges versus minimum non-negative weighted cost.

Is DFS shortest?

Answer: No; its traversal order chooses a valid path.

Bellman-Ford negative-cycle test?

Answer: A relaxation after V-1 passes indicates reachable negative cycle.

Topology vs scenario?

Answer: Structure versus one experiment configuration.

What does seed do?

Answer: Selects pseudo-random sequence for repeatability.

Observer pattern?

Answer: Engine events notify independent trace/metrics listeners.

Time-weighted queue?

Answer: Integrate queue size over time and divide by duration.

Why GroupKFold by topology?

Answer: Avoid leakage among windows from same graph.

What does baseline flag mean?

Answer: Deviation from expected, not cause/proven fault.

What does GraphRAG add?

Answer: Graph relationships link evidence across run entities.

How constrain LLM?

Answer: Bound evidence, cite retrieved IDs, validate, abstain.

What is 413?

Answer: Prompt/request too large; retrieve/aggregate and cap evidence.

How GUI independent from CLI?

Answer: Structured API and exported files, not terminal text.

Why strict CSV schema?

Answer: Shared contract prevents false/missing data.

How test reroute after failure?

Answer: Dual path, fixed seed, scheduled outage, inspect table, trace, metrics.

Limitations?

Answer: Model approximates network; ML depends on data; LLM depends on evidence.

21. Glossary and source map

Term	Meaning
Adjacency list	Direct neighbors/links for each node.
Channel	One-direction runtime queue on a Link.
Discrete event	Clock jumps to next scheduled state change.
Epoch	Counter invalidating callbacks from old state.
Goodput	Useful delivered data rate.
Knowledge graph	Typed nodes and edges for entities/relationships.
Manifest	Run metadata tying settings to files.
Observer	Listener notified of events/state changes.
P95	Value at or below 95% of measured latencies.
TTL	Hop limit decremented by routers.

Topic	Repository location
C++ common/model	include/netmind/common/, include/netmind/model/, src/model/
Graph	include/netmind/graph/, src/graph/, config/topologies/
Routing	include/netmind/routing/, src/routing/
Simulation	include/netmind/simulation/, src/simulation/
Metrics/CLI	include/netmind/metrics/, src/metrics/, include/netmind/cli/, src/cli/
Python	python/common/, python/ml/, python/graphrag/, python/netmind_api.py
Frontend/replay	frontend_v2/src/, frontend_v2/public/legacy/
Contracts/tests	docs/contracts/, tests/, python/tests/
Plan/roles	Project901 - Phase-wise Implementation Guide (v2).md; TEAM_RESPONSIBILITIES.md

NetMind combines graph algorithms, OOP modeling, discrete-event simulation, metrics, reproducible experiments, expected-value ML, and graph-grounded answers. Tell the story through evidence: configuration -> simulation -> trace/metrics -> baseline/replay/GraphRAG.